A ONE-HOUR TECHNICAL SESSION FOR RF & RAN ENGINEERS

Deep Reinforcement Learning for RAN Planning & Optimization

How DQN and PPO learn to tune coverage, capacity, mobility, interference, and energy the way an experienced RF engineer would — explained without the math.

Dr Rao S Yenamandra · DigitalTwinSim

60 MINUTES · 29 TOPICS · LIGHT ON MATH, HEAVY ON INTUITION

What We'll Cover in 60 Minutes

00–10 min	Why RAN optimization needs AI now, and what RL actually is
10–25 min	DQN and PPO explained simply, with RAN use cases for each
25–45 min	Deep dives: coverage & capacity, mobility, energy, interference, planning
45–55 min	How training really works, risks, guardrails, and a decision guide
55–60 min	Takeaways and open discussion

Why AI Is Entering RAN Now

- **Complexity has outgrown manual tuning.** Multi-band, multi-vendor, massive MIMO, and dense small cells create far more tunable parameters than a team can hand-tune.

- **Rule-based SON hits a ceiling.** Static thresholds handle known patterns, but not the combinatorial interactions across thousands of cells.

- **5G/6G add new control dimensions.** Beamforming, energy targets, network slicing, and sensing all need coordinated, not isolated, tuning.

SCALE OF THE PROBLEM

1000s

of cells per market, each with dozens of tunable parameters

24/7

traffic patterns that shift hourly, daily, and seasonally

Multi-cell

tradeoffs a single-cell fix can't see (coverage vs. interference)

The Limits of Traditional RAN Optimization

- **Reactive, not consequential.** Rule-based SON and legacy ML respond to known patterns — they don't learn the downstream effect of an action.

- **Blind to delayed tradeoffs.** Traditional tools optimize what they can see right now, not what happens two rings of cells later.

A FAMILIAR EXAMPLE

You boost power to fix a coverage hole. SINR improves locally — problem solved, according to the rule. But two rings of cells out, interference quietly rises, degrading throughput for neighbors the rule never looked at.

What Is Reinforcement Learning? (Plain-Language Loop)

Agent	The optimization controller making decisions
Environment	The live or simulated RAN — cells, UEs, traffic
State	What it observes: SINR, PRB load, throughput, latency, handovers
Action	A parameter it changes: tilt, power, HO offset, sched. weight
Reward	A score for network health: throughput ↑, drops ↓, energy ↓

The loop: observe → act → get scored → learn → repeat — thousands of times in simulation before ever touching a live network.

The RL Loop, in a Few Simple Equations

State	$s_t = (SINR_t, PRB_t, Thpt_t, Lat_t, \dots)$	A snapshot of network conditions the agent observes at time t
Action	DQN: $a_t \in \{a_1, \dots, a_n\}$ | PPO: $a_t \in [a_{min}, a_{max}]$	A discrete pick (DQN) or a precise continuous value (PPO)
Reward	$r_t = w_1 \cdot Throughput_t - w_2 \cdot Drops_t - w_3 \cdot Energy_t$	A weighted KPI score — same idea as a weighted scorecard engineers already use
Penalty	$r_t' = r_t - \lambda \cdot Violation(a_t)$	Subtracts a cost whenever an action breaks an operating rule
Guardrail	$a_t = clip(a_t , a_{min} , a_{max})$	Hard-clamps the action so it can never leave a safe operating range
Objective	$G_t = \Sigma \ \gamma^k \cdot r_{t+k} \quad (k = 0 \to \infty)$	What the agent is really maximizing: reward now plus discounted future reward

None of this runs by hand — the network learns the weights and bounds through simulation. This is just the shape of what it's learning.

The RAN as a Multi-Step Decision Process (MDP)

An MDP is just a formal name for “a sequence of decisions where each one shapes the next.” Formally: (S, A, P, R, γ) — states, actions, transitions, rewards, and a discount factor.

TWO IDEAS THAT MATTER IN PRACTICE

- **Markov property:** s_{t+1} depends only on s_t and a_t — the agent doesn't need the network's entire history, just its current reading.
- **Credit assignment:** a tilt or power change now may not show its full effect for several steps — handover patterns and interference settle over minutes. γ (from the equations slide) is what lets the agent value that delayed payoff.

Simple RL vs. Deep RL

Simple (Tabular) RL

- Keeps a lookup table: one best action per state
- Works only for small, discrete problems
- Examples: Q-learning, SARSA, Monte Carlo
- Breaks down once state space gets large

Deep RL

- Replaces the table with a neural network
- Generalizes across states it hasn't exactly seen
- Examples: DQN, PPO, and others
- Needed the moment state is large or continuous — which is basically always in real RAN data

Two Families of DRL Algorithms

DISCRETE CHOICES

DQN

Picks the best option from a fixed menu of actions

CONTINUOUS CONTROL

PPO

Outputs a precise value and updates behavior gradually and safely

Both learn from trial and error in simulation — they differ in the shape of the action they produce.

DQN Explained Simply

DQN (Deep Q-Network) learns to answer one question: “Given this network state, which of my available discrete actions has historically led to the best long-term outcome?”

BEST SUITED TO

- Categorical, discrete decisions — not fine continuous values
- A fixed, enumerable menu of choices at each step
- Situations where 'which one' matters more than 'how much'

DQN Use Cases in RAN Planning & Optimization

Cell sleep mode

Turn a carrier or cell ON/OFF for energy saving in low traffic

Beam index selection

Choose one of a fixed set of beamforming codebook entries

Discrete power steps

Pick a power level from a preset list (e.g., 5 fixed steps)

Admission control

Admit, queue, or reject a new session

Handover target

Select which neighboring cell to hand a UE to

Scheduling tier

Choose a discrete MCS / priority tier

DQN: Choosing Inputs and Outputs

INPUTS (STATE FEATURES)

- PRB utilization (%) for this cell and neighbors
- Active UE count and traffic mix
- Time of day / day-of-week traffic trend
- Current SINR and interference readings

OUTPUT — ONE Q-VALUE PER DISCRETE ACTION

Example action menu: {Stay Awake, Sleep 1 Carrier, Sleep 2 Carriers}

```
Q = [ 0.82, 0.91, 0.34 ]
```

Agent picks the highest-scoring action

```
action = argmax(Q)  →  Sleep 1 Carrier
```

Design choice for DQN: the input is a feature vector; the output layer has exactly one node per possible action. Adding a new action means adding a new output node and retraining.

PPO Explained Simply

PPO (Proximal Policy Optimization) outputs a continuous number directly — the exact tilt, exact power in dBm, exact resource split — rather than choosing from a menu.

WHY “PROXIMAL” MATTERS

- It constrains how much the policy can change in one update
- Stays close to the last known-good behavior — no erratic swings
- Critical in live networks, where a bad update has real consequences

BEST SUITED TO

- Continuous, fine-grained values — not a fixed menu of choices
- Smooth, gradual adjustment toward a target setting
- Situations where 'how much' matters more than 'which one'

Weighting the PPO Reward by KPI Importance

PPO doesn’t optimize each KPI on its own — it maximizes one weighted score. Every input KPI carries a weight set by its importance to the slice SLA, so re-weighting the reward re-points the same agent at a new objective.

EXAMPLE — eMBB REWARD WEIGHTS

Throughput — w ≈ 0.35 · primary eMBB SLA target

PRB / spectral efficiency — w ≈ 0.25 · capacity headroom

SINR margin — w ≈ 0.20 · link quality

Energy per bit — w ≈ 0.12 · OPEX

Latency — w ≈ 0.08 · eMBB tolerates delay — weighted last

Weights are chosen in reward design (see slide 6) — set by engineers, not learned.

SAME KPIs, DIFFERENT SLICE

- URLLC — latency and reliability jump to the top; throughput weight drops down the list
- mIoT — admission fairness and energy dominate; throughput weight falls near zero
- The weights encode the SLA — the state and action design never change

PPO Use Cases in RAN Planning & Optimization

Antenna tilt (CCO)

Continuous tilt optimization for coverage & capacity

Transmit power control

Fine-grained per-cell / per-beam power adjustment

Massive MIMO beams

Beam weight and beam-steering optimization

Energy scaling

Continuous power/PRB scaling with load, not hard on/off

Mobility (MRO)

Tuning HO margins, time-to-trigger, hysteresis

Load balancing

Weights across carriers, bands, or RATs (5G/Wi-Fi)

PPO: Choosing Inputs and Outputs

INPUTS (STATE FEATURES)

- SINR distribution across the sector
- Throughput, latency, and drop-rate trends
- Interference reported by neighboring cells
- Current tilt / power setting (so it knows its starting point)

OUTPUT — A DISTRIBUTION OVER A CONTINUOUS VALUE

The network outputs a mean and a spread, not one fixed number

$$\mu = 4.2°, \quad \sigma = 0.3°$$

Training samples from it (exploration); inference just takes the mean

$$a_t \sim N(\mu, \sigma) \rightarrow a_t = \mu$$

In plain terms: μ is the value the policy settled on, and σ is how far it explores around μ while training — at deployment it just uses μ.

Design choice for PPO: the output layer has two nodes per continuous action (its mean and spread) instead of one node per menu item — that's what lets it express "tilt down by 4.2°" instead of only choosing from a fixed list.

Custom Neural Network vs. Open-Source SLM as the Policy Backbone

Custom Neural Network

- Small MLP/CNN trained from scratch directly on numeric KPI vectors
- Thousands to low millions of parameters
- Inference in microseconds — fits inline in a real-time control loop
- Trained purely from the reward signal (DQN/PPO) — no language understanding

Open-Source SLM

- Pretrained small language model (e.g. Llama, Phi, Mistral-class) repurposed as a decision layer
- Billions of parameters — orders of magnitude larger
- Inference in tens to hundreds of milliseconds — too slow for a tight per-TTI loop
- Brings general reasoning and can explain its decision in plain language

Rule of thumb: use a custom NN for the fast, precise, numeric control loop itself (tilt, power, admission, sleep). An SLM fits better one layer up — reading alarms/tickets in natural language and deciding which specialized DQN/PPO policy to invoke, or explaining a decision to an engineer.

PART THREE

Five Use Cases, Deep Dive

Coverage · Mobility · Energy · Interference · Planning

Coverage & Capacity Optimization (CCO)

TYPICALLY PPO

The agent learns to balance coverage holes against overshoot interference across a whole cluster of cells — something single-cell tilt optimization can't do, because it never sees the multi-cell coupling that continuous tilt and power changes create.

Mobility Robustness Optimization (MRO)

TYPICALLY PPO

Reduces ping-pong handovers, too-early/too-late handovers, and radio link failures by learning handover parameters per cell-pair from real mobility patterns — instead of one fixed threshold applied everywhere.

Energy Savings

DQN + PPO

Balances energy consumption against QoS by learning when to sleep carriers/cells (DQN) or continuously scale massive MIMO power (PPO) — adapting to each site's actual daily and weekly traffic rhythm rather than a fixed time-based schedule.

Interference Management

TYPICALLY PPO

Learns per-cell power and PRB allocation patterns that reduce inter-cell interference in dense deployments — especially relevant for small-cell clusters and C-band rollouts where classical ICIC struggles to coordinate across many neighbors.

Network Planning Assistance

SIMULATION-DRIVEN

DRL agents trained inside digital twins/simulators rapidly evaluate ‘what if’ site placement, tilt, and power configurations before rollout — compressing planning cycles that used to require manual, iterative drive-testing.

6G: What Changes, What Doesn't

- **Extreme massive MIMO / sub-THz** Beam management at a scale manual tuning can't reach
- **Integrated Sensing & Communication (ISAC)** Joint control of sensing and communication parameters
- **Reconfigurable Intelligent Surfaces (RIS)** Phase-shift optimization across surface elements

All three are naturally continuous control problems — a strong fit for PPO-family methods.

Training Phase vs. Inference Phase

Training Phase

- Agent explores — takes some random / noisy actions on purpose
- Experience is collected: a replay buffer (DQN) or rollout trajectories (PPO)
- A loss is computed (TD-error for DQN, clipped surrogate objective for PPO) and weights are updated by backpropagation
- Runs for many simulated episodes — hours to days, on GPU compute, entirely offline

Inference Phase

- Weights are frozen — no more learning happens on the fly
- Exploration is switched off: the agent takes its best-known action (argmax for DQN, the mean μ for PPO)
- Just one forward pass through the network per decision
- Runs in milliseconds on modest compute — suitable for a live RIC control loop

How Do You Know It Converged?

Convergence = training has stopped meaningfully improving the policy.

- **Reward curve flattens.** Episode return climbs, then plateaus instead of still trending up.
- **Loss stabilizes.** TD-error (DQN) or policy loss (PPO) decreases and levels off near a small value.
- **Policy stops changing much between updates.** PPO's own clipping is built around bounding this step-to-step change.
- **Validation KPIs hold steady across independent runs.** Simulated throughput / drop-rate / energy stop drifting run to run.

Watch the curve, don't trust a fixed iteration count — a flat reward can also mean the agent settled into a local optimum, not the best possible policy. Validate against varied traffic scenarios before trusting it.

How Training Actually Works

1. Simulate

Train offline in a simulator or digital twin using historical or synthetic traffic — never directly on the live network

2. Validate

Extensive testing in simulation before any real-world exposure

3. Shadow mode

Deployed live but only observing and recommending — not acting

4. Canary rollout

Small cluster deployment, engineers monitoring with rollback ready

Human engineers stay in the loop at every stage, with authority to roll back at any point.

Risks and Practical Guardrails for RF Engineers

- **Reward shaping pitfalls** A poorly designed reward can optimize the wrong thing — e.g., maximizing throughput while ignoring energy or fairness
- **Hard safety bounds** Guardrails so the agent can't take extreme actions, like power hard limits
- **Explainability** Engineers need visibility into why an action was taken, not a black box
- **Fallback to rules** Revert to rule-based control if the agent behaves unexpectedly
- **Drift monitoring** Watch for performance decay as traffic patterns change over time

Guarded DQN vs. Guarded PPO — a Real PRACH Storm Result

PRACH storm mitigation: synchronized mIoT devices flood the random-access channel, collide on preambles, and retry — a real overload scenario, not a toy problem.

THE SHARED GUARD

`Admission ceiling ∝ 0.90M/B_t` caps how wide either agent can open access | `Recovery floor ∝ 0.35M/B_t` stops either agent from throttling forever while idle

	Guarded DQN	Guarded PPO
Success rate (3,000-device storm)	100%	99.8%
Failures	0	7
95th-pct. delay	18.6 s	22.2 s
Held-out scenarios won (of 30)	20	0

Read carefully: both controllers shared the identical guard, state, reward, and training storms — this favors DQN within this shared guard on a 13-action discrete PRACH control problem, not a general claim that DQN beats PPO. Guard-only and unguarded ablations are still future work.

DQN vs. PPO — Quick Reference

	DQN	PPO
Control variable	Discrete / categorical	Continuous / fine-grained
Example	Which cell, which beam index, on/off	Exact power, exact tilt, exact ratio
Update style	Selects from a fixed menu	Smooth, bounded, gradual updates
Best fit for	Simpler sandbox problems first	Production-adjacent, safety-critical rollout

Key Takeaways

- **DRL is disciplined trial-and-error, not magic.** It outperforms static rules in complex, multi-cell, multi-objective RAN environments.
- DQN suits discrete decisions; PPO suits continuous, fine-grained, safer control.
- Success depends more on reward design and staged rollout than on algorithm choice alone.
- **Start bounded.** Pick one well-instrumented use case — energy savings or tilt optimization — before attempting network-wide control.

OPEN DISCUSSION

Questions & Discussion

Which of your current manual tuning tasks looks most like a DQN problem — and which looks more like a PPO problem?